



NOMBRES Y MATRICULAS

Jhonatan Eliseo Romero Jacinto || 20242685

Jonathan Joel Geraldo Frias || 20242604

Daniel Ureña Sanchez || 20231328

MATERIA

Programación Paralela 2026-C-2

PROFESOR

Erick Leonardo Pérez Veloz

TEMA

Minería Blockchain: nodo minero para una cooperativa de energía solar

FECHA

12/ 8 /2026

1. Índice

1.	Índice	2
2.	Introduccion	4
2.1.	Presentacion del Proyecto.....	4
2.2.	Justificación del tema elegido	4
2.3.	Objetivos	5
	Objetivo general	5
	Objetivos específicos	5
3.	Descripcion del problema	5
3.1.	Problema Seleccionado	5
3.2.	Aplicación en un escenario real.....	6
3.3.	Importancia del paralelismo en el proyecto	6
4.	Cumplimiento de los requisitos del proyecto.....	6
	Requisito 1 - Ejecución simultánea de múltiples tareas.....	6
	Requisito 2 - Necesidad de compartir datos entre tareas.....	7
	Requisito 3 - Exploración de diferentes estrategias de paralelización	7
	Requisito 4 - Escalabilidad con más recursos.....	7
	Requisito 5 - Métricas de evaluación del rendimiento.....	7
	Requisito 6 - Aplicación a un problema del mundo real	7
5.	Diseño de la solución	8
5.1.	Arquitectura general del Sistema	8
5.2.	Diagrama de componentes y tareas paralelas	9
5.3.	Estrategia de paralelización usada.....	9
5.4.	Herramientas y tecnologías usadas	9
6.	Implementacion técnica	10
6.1.	Estructura del proyecto.....	10
6.2.	Explicacion del código clave	10
6.3.	Usos de mecanismos de sincronizacion	11
6.4.	Justificacion de técnicas de las decisiones	11
7.	Evaluacion de desempeño	12
7.1.	Comparativa entre ejecución secuencial y paralela	12

7.2.	Métricas: tiempo, eficiencia y escalabilidad	13
7.3.	Análisis de cuellos de botella y limitaciones	14
	Análisis por dificultad	14
8.	Trabajo en Equipo	15
8.1.	Reparto de tareas	15
8.2.	Herramientas de coordinación	16
9.	Conclusiones.....	16
9.1.	Principales aprendizajes técnicos	16
9.2.	Retos enfrentados y superados	16
9.3.	Posibles mejoras o líneas futuras	16
10.	Referencias	17
11.	Anexos	17
11.1.	Manual de ejecución del Sistema.....	17
11.2.	Capturas y pruebas complementarias	18
11.3.	Enlace al repositorio.....	19

2. Introduccion

2.1. Presentacion del Proyecto

El proyecto consiste en el desarrollo de un nodo minero para una cooperativa de energía solar: un sistema que registra de forma confiable e inalterable los intercambios de energía entre las viviendas de una comunidad, utilizando una blockchain (cadena de bloques). El componente central del sistema es el proceso de sellado de bloques (minería), que se implementa de forma paralela para demostrar la ventaja de ejecutar el trabajo entre varios hilos del procesador frente a hacerlo con uno solo.

El sistema se desarrolló en C# sobre la plataforma .NET 8, empleando la Task Parallel Library (TPL) para el paralelismo. La técnica de paralelización aplicada es la descomposición exploratoria, adecuada para problemas de búsqueda donde existe un espacio enorme de posibilidades que debe recorrerse hasta encontrar una solución válida.

2.2. Justificación del tema elegido

La minería de una blockchain es un caso ideal para demostrar la descomposición exploratoria por tres razones. Primero, el sellado de un bloque no tiene fórmula ni atajo: obliga a probar millones de valores (nonces) hasta encontrar uno válido, lo que constituye un espacio de búsqueda divisible entre varios trabajadores. Segundo, cada intento es independiente de los demás, lo que permite repartir la búsqueda y obtener una aceleración casi directa al añadir núcleos. Tercero, el problema exige compartir datos y sincronizar: cuando un hilo encuentra la solución, debe avisar a los demás para que se detengan, lo que ejercita los mecanismos de sincronización.

Además, el tema tiene una aplicación real y actual, y es que los registros energéticos comunitarios sobre blockchain existen, lo que permite anclar el proyecto a un escenario concreto y demostrar su utilidad más allá de lo académico.

2.3. Objetivos

Objetivo general

Demostrar la ventaja del paralelismo y el manejo de sus complejidades (sincronización de datos, eficiencia y escalabilidad) mediante la implementación paralela del sellado de bloques de una blockchain aplicada a una cooperativa de energía solar.

Objetivos específicos

- Implementar el sellado de bloques (Proof-of-Work) en versión secuencial y paralela para poder compararlas.
- Repartir el espacio de búsqueda de nonces entre varios hilos mediante la Task Parallel Library.
- Coordinar los hilos con mecanismos de sincronización que detengan la búsqueda al encontrar la solución, evitando condiciones de carrera.
- Medir el rendimiento (tiempo, aceleración, eficiencia) al variar el número de hilos y la dificultad, y analizar la escalabilidad.
- Construir la cadena completa de bloques y verificar su integridad, demostrando cómo se detecta cualquier alteración de los datos sellados.
- Aplicar el sistema a un escenario del mundo real: el registro de energía de una cooperativa solar, alimentado por reportes en formato CSV.

3. Descripción del problema

3.1. Problema Seleccionado

Una blockchain es un registro digital compartido en el que, una vez que un dato se anota y se sella, ya no puede alterarse sin que se detecte. Los datos se agrupan en bloques, y cada bloque queda encadenado al anterior mediante una huella digital (hash). Sellar un bloque (lo que se conoce como minar) consiste en encontrar un número especial, el nonce, tal que la huella del bloque cumpla una condición de dificultad (empezar con cierta cantidad de ceros).

El reto computacional está en que no existe forma de calcular ese nonce directamente: la única vía es probar valores uno tras otro hasta acertar. Con dificultades altas, esto implica millones de intentos, lo que convierte al sellado en una tarea costosa en tiempo. Este es precisamente el tipo de problema (una búsqueda exhaustiva sobre un espacio inmenso) donde el paralelismo aporta una ventaja clara.

3.2. Aplicación en un escenario real

El escenario elegido es una cooperativa de energía solar: una comunidad de viviendas con paneles solares donde unas casas producen más energía de la que consumen y otras menos. La comunidad necesita un registro confiable e imposible de manipular de cuánta energía aporta y consume cada vivienda, para repartir costes y beneficios con transparencia y sin que nadie pueda hacer trampa con los números.

La blockchain resuelve esa necesidad: cada intercambio de energía (por ejemplo, “Casa-03 aportó 4.5 kWh”) se registra como una transacción, las transacciones se agrupan en bloques y se sellan. A partir de ese momento, alterar cualquier dato rompe la cadena y queda en evidencia. Este tipo de sistema existe en la realidad; un ejemplo conocido es el proyecto de micro-red energética Brooklyn Microgrid. En un despliegue completo, los datos vendrían firmados desde medidores inteligentes instalados en cada vivienda; en este proyecto se simulan mediante reportes en CSV para concentrar el esfuerzo en el componente de sellado paralelo.

3.3. Importancia del paralelismo en el proyecto

El sellado es el cuello de botella del sistema: si tarda demasiado, las transacciones de energía se acumulan sin confirmar. Como la búsqueda del nonce consiste en millones de intentos independientes, se puede repartir entre varios núcleos que buscan simultáneamente en zonas distintas del espacio de nonces. Con N trabajadores se cubre el espacio aproximadamente N veces más rápido.

Esta es la esencia de la descomposición exploratoria: no se divide una única tarea en pasos, sino que se divide el espacio de posibilidades de una búsqueda, y varios trabajadores lo exploran a la vez hasta que uno encuentra la solución y avisa a los demás. El proyecto demuestra numéricamente esa ventaja comparando el tiempo de sellado con uno y con varios hilos.

4. Cumplimiento de los requisitos del proyecto

Requisito 1 - Ejecución simultánea de múltiples tareas

El sellado del bloque reparte el rango de nonces entre varios hilos que se ejecutan a la vez. Cada hilo prueba un subconjunto distinto de nonces mediante partición por intercalado: con N hilos, el hilo 0 prueba los nonces 0, N , $2N$...; el hilo 1 prueba 1, $N+1$, $2N+1$..., y así sucesivamente. De este modo, N búsquedas ocurren de forma simultánea en lugar de una sola secuencial. Los hilos se lanzan con “Task.Run” de la Task Parallel Library y se esperan con “Task.WaitAll”.

Requisito 2 - Necesidad de compartir datos entre tareas

Los hilos comparten varios datos. Todos leen la misma parte fija del bloque (transacciones, hash previo, dificultad). Además, comparten una señal de parada (“CancellationToken”): cuando un hilo encuentra un nonce válido, la activa y todos los demás se detienen, porque seguir buscando ya no tiene sentido. También comparten un contador global del total de hashes probados. Estas escrituras compartidas exigen sincronización, que se resuelve con “Interlocked” (operaciones atómicas) para evitar condiciones de carrera. Esta es la parte del proyecto que ejercita directamente la sincronización de datos.

Requisito 3 - Exploración de diferentes estrategias de paralelización

Descomposición exploratoria

Requisito 4 - Escalabilidad con más recursos

Al añadir más hilos, la velocidad de sellado (hashes por segundo) aumenta y el tiempo para sellar un bloque disminuye. El proyecto mide este comportamiento ejecutando el sellado con 1, 2, 4 y 8 hilos, lo que permite observar cómo escala el rendimiento con los recursos disponibles. Se observa la escalabilidad típica del paralelismo: la aceleración crece al añadir hilos, con rendimientos decrecientes a partir del número de núcleos físicos del procesador.

Requisito 5 - Métricas de evaluación del rendimiento

El proyecto calcula y exporta a un archivo CSV las siguientes métricas: velocidad en millones de hashes por segundo (MHash/s), tiempo de sellado, aceleración (speedup - cuántas veces más rápido va la versión paralela frente a la secuencial) y eficiencia (qué porcentaje de aprovechamiento aporta cada hilo).

Requisito 6 - Aplicación a un problema del mundo real

Imaginemos una cooperativa de, digamos, 5 vecinos con paneles solares. Durante el día, unas casas producen más energía de la que gastan (la que sobra la mandan a una red compartida), y otras consumen más de lo que producen (toman de esa red). Al final del mes hay que responder una pregunta espinosa: ¿quién le debe cuánto a quién?

Aquí está el conflicto: si cada uno apunta sus propios números, nadie confía en nadie. El vecino que aportó mucho quiere que se le reconozca, y sospecha que otro está inflando lo que produjo para pagar menos. Sin un árbitro confiable, esto se vuelve un problema de confianza. Y esto es lo que resuelve.

Cómo funcionaría

- **Paso 1 — Se mide.** Cada casa tiene un medidor que registra automáticamente cuánta energía aporta y consume. En el proyecto, son los CSV de energía (el reporte por período). Pero realmente vendría de un medidor digital.
- **Paso 2 — Se sella.** Cada cierto tiempo (una semana, una quincena, un mes), esos datos se agrupan y se sellan en la blockchain como un bloque. Este es el corazón del proyecto, y el sellado se hace en paralelo para que sea rápido. A partir de ahí, esos números quedan congelados e imposibles de alterar.
- **Paso 3 — Se confía.** Como el registro es inalterable y todos pueden verlo. Nadie puede volver atrás y cambiar "aporté 4 kWh" por "aporté 40 kWh", porque romper el sello se detecta al instante.

Que haría una corporativa con estos datos:

- **Repartir el dinero de forma justa.** Al final del período, se abre la cadena y se suma: cuánto aportó cada casa, cuánto consumió. Con eso se calcula quién debe pagar y quién debe recibir. Como los números están sellados, la factura es indiscutible. Nadie puede reclamar que los datos fueron manipulados.
- **Resolver disputas.** Si un vecino dice "yo aporté más de lo que dice mi factura", se va a la cadena, se revisa el bloque de ese período, y se comprueba. La cadena es la fuente de verdad, el árbitro neutral.
- **Rendir cuentas con transparencia.** Cualquier miembro puede auditar el registro completo cuando quiera. No hay una empresa central que controle los números a puerta cerrada; el registro es compartido y verificable por todos.
- **Tener un histórico confiable.** Con el tiempo, la cadena guarda el historial completo de la energía de la comunidad, útil para decidir si conviene instalar más paneles, cómo evoluciona el consumo, etc.

5. Diseño de la solución

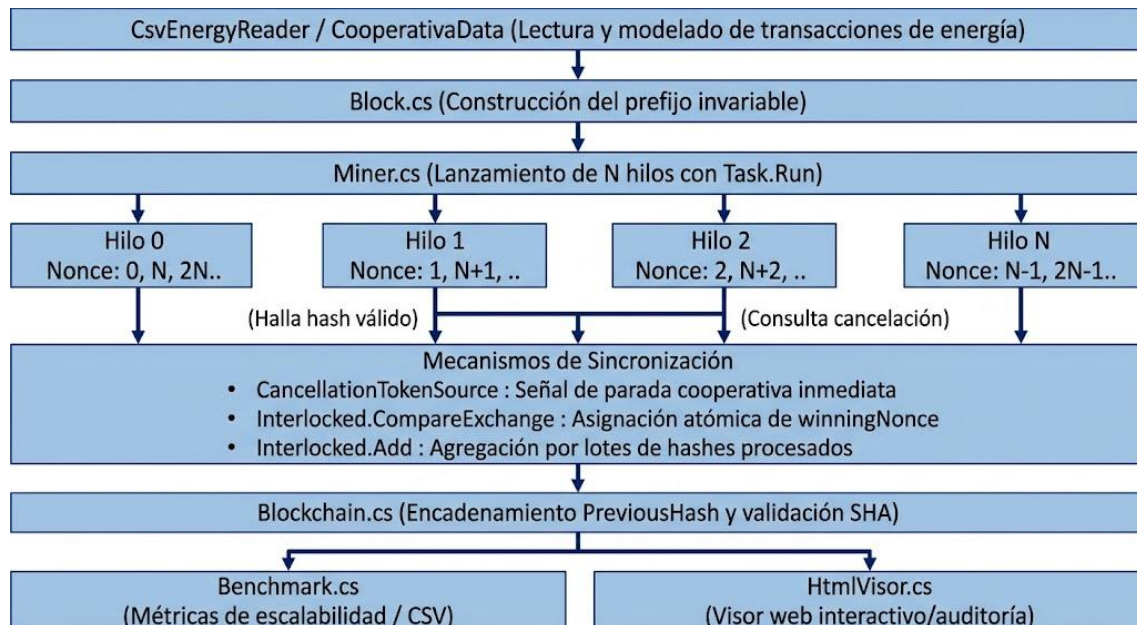
5.1. Arquitectura general del Sistema

El sistema sigue un flujo de tres fases que reproduce, de forma simplificada, cómo funcionaría una plataforma real de registro energético:

1. **Fase de datos (entrada):** se generan reportes de energía de las viviendas en formato CSV (lo que en un sistema real provendría de los medidores inteligentes).

2. **Fase de sellado (procesamiento paralelo):** el programa lee los CSV y sella un bloque por cada período, buscando el nonce en paralelo. Es el núcleo del proyecto.
3. **Fase de verificación (salida):** la cadena se exporta a JSON y a un visor HTML interactivo que permite comprobar la integridad y detectar cualquier alteración.

5.2. Diagrama de componentes y tareas paralelas



5.3. Estrategia de paralelización usada

La estrategia es la descomposición exploratoria con partición por intercalado. El espacio de nonces se reparte entre los hilos de forma que cada uno recorre su propio “carril” de números sin solaparse con los demás, y entre todos cubren el espacio completo. Un “CancellationToken” compartido actúa como señal de parada global: el primer hilo que encuentra un nonce válido la activa y detiene a los demás. El conteo de intentos se agrega de forma segura con operaciones atómicas “Interlocked”.

5.4. Herramientas y tecnologías usadas

Herramienta	Uso en el proyecto
C# / .NET 8	Lenguaje y plataforma de desarrollo.
Task Parallel Library (TPL)	Paralelismo: Task.Run y Task.WaitAll para lanzar y coordinar los hilos.
System.Security.Cryptography	Cálculo del hash SHA-256 (la huella de cada bloque).
Interlocked / CancellationToken	Sincronización: parada compartida y conteo atómico.

Stopwatch	Medición precisa de los tiempos de sellado.
HTML + JavaScript (Web Crypto)	Visor interactivo que recalcula hashes en el navegador para verificar la cadena.
Git / GitHub	Control de versiones y trabajo en equipo.
Excel	Elaboración de las gráficas a partir del CSV de métricas.

6. Implementacion técnica

6.1. Estructura del proyecto

El código se organiza en módulos con una responsabilidad clara cada uno:

Módulo	Responsabilidad
EnergyTransaction	Representa un intercambio de energía (vivienda, tipo, kWh, fecha).
Block	Un bloque: agrupa transacciones, calcula su hash y guarda el del bloque anterior.
Miner	El núcleo: sellado secuencial y paralelo del bloque (búsqueda del nonce).
Blockchain	Arma la cadena de bloques encadenados y verifica su integridad.
CsvEnergyReader	Lee los CSV de energía y los convierte en transacciones.
ReportCsvWriter / ReportGenerator	Generan los reportes de energía por período (CSV y página HTML).
HtmlVisor	Genera el visor HTML interactivo que detecta alteraciones.
Benchmark	Mide el rendimiento con distinto número de hilos y exporta el CSV de métricas.
Program	Integra todo y ofrece los modos: benchmark, chain, reporte y sellar.

6.2. Explicacion del código clave

El corazón del sistema es el bucle de sellado. En su forma paralela, cada hilo ejecuta un bucle que, para cada nonce de su carril, construye el contenido del bloque, calcula su hash SHA-256 y comprueba si cumple la dificultad. Si la cumple, activa la señal de parada; si no, salta a su siguiente nonce. Un aspecto clave de la implementación es que la parte fija del bloque se calcula una sola vez (no en cada intento), y la comprobación de los ceros se hace directamente sobre los bytes del hash, evitando convertirlo a texto millones de veces. Estas dos optimizaciones aceleran notablemente el sellado.

```

public MiningResult MineParallel(Block block, int difficulty, int threadCount)
{
    var stopwatch = Stopwatch.StartNew();
    using var cts = new CancellationTokenSource();
    var token = cts.Token;

    byte[] prefix = block.GetContentPrefixBytes();

    long totalHashesTried = 0;
    long winningNonce = -1;

    var tasks = new Task[threadCount];
    for (int t = 0; t < threadCount; t++)
    {
        int offset = t;
        tasks[t] = Task.Run(() =>
        {
            byte[] buffer = new byte[prefix.Length + MaxNonceDigits];
            Array.Copy(prefix, buffer, prefix.Length);
            Span<byte> hash = stackalloc byte[32];

            long localHashes = 0;
            long nonce = offset;

            while (!token.IsCancellationRequested)
            {
                Utf8Formatter.TryFormat(nonce, buffer.AsSpan(prefix.Length), out int written);
                SHA256.HashData(buffer.AsSpan(0, prefix.Length + written), hash);
                localHashes++;

                if (MeetsDifficultyBytes(hash, difficulty))
                {
                    Interlocked.CompareExchange(ref winningNonce, nonce, -1);
                    cts.Cancel();
                    break;
                }

                nonce += threadCount;
            }

            if ((localHashes & 0x3FFF) == 0)
            {
                Interlocked.Add(ref totalHashesTried, localHashes);
                localHashes = 0;
            }
        });
    }
}

```

```

        Interlocked.Add(ref totalHashesTried, localHashes);
    });
}

Task.WaitAll(tasks);
stopwatch.Stop();

block.Nonce = winningNonce;
block.Hash = block.ComputeHash();
return new MiningResult
{
    Block = block,
    Nonce = winningNonce,
    Hash = block.Hash,
    ElapsedSeconds = stopwatch.Elapsed.TotalSeconds,
    HashesTried = totalHashesTried,
    ThreadUsed = threadCount,
    Difficulty = difficulty
};
}

```

6.3. Usos de mecanismos de sincronización

Se emplean dos mecanismos. El “CancellationToken” funciona como una bandera compartida de “ya se encontró”: un hilo la activa y todos consultan su estado en cada vuelta del bucle para detenerse. “Interlocked” realiza operaciones atómicas (sumar al contador global y fijar el nonce ganador) garantizando que, aunque varios hilos escriban a la vez, no se corrompa el dato ni se pierdan actualizaciones. Estos mecanismos resuelven las condiciones de carrera propias del trabajo compartido.

6.4. Justificación de técnicas de las decisiones

Se eligió “Task.Run” sobre “Parallel.For” porque el espacio de búsqueda es indefinido: no se sabe de antemano cuántos nonces habrá que probar (podrían ser miles o millones), y “Parallel.For” está pensado para rangos finitos conocidos. Se construyó una blockchain propia y ligera en lugar de usar una plataforma existente porque así el paralelismo queda visible y medible, que es lo que la asignatura evalúa. Finalmente, se fijó un “timestamp” constante en las comparaciones para garantizar que las versiones secuencial y paralela resuelvan exactamente el mismo problema, haciendo la comparación justa.

7. Evaluacion de desempeño

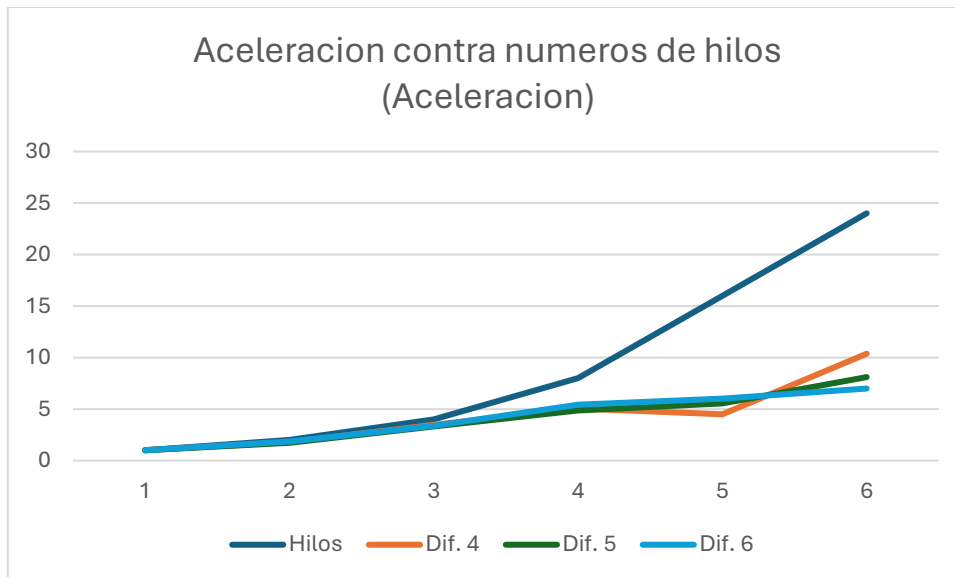
7.1. Comparativa entre ejecución secuencial y paralela

Para medir la ventaja del paralelismo se selló el mismo bloque con distinto número de hilos y distintas dificultades. La versión de un hilo actúa como línea base (secuencial) y las demás como versiones paralelas. La aceleración (speedup) indica cuántas veces más rápido va cada versión paralela respecto a la secuencial.

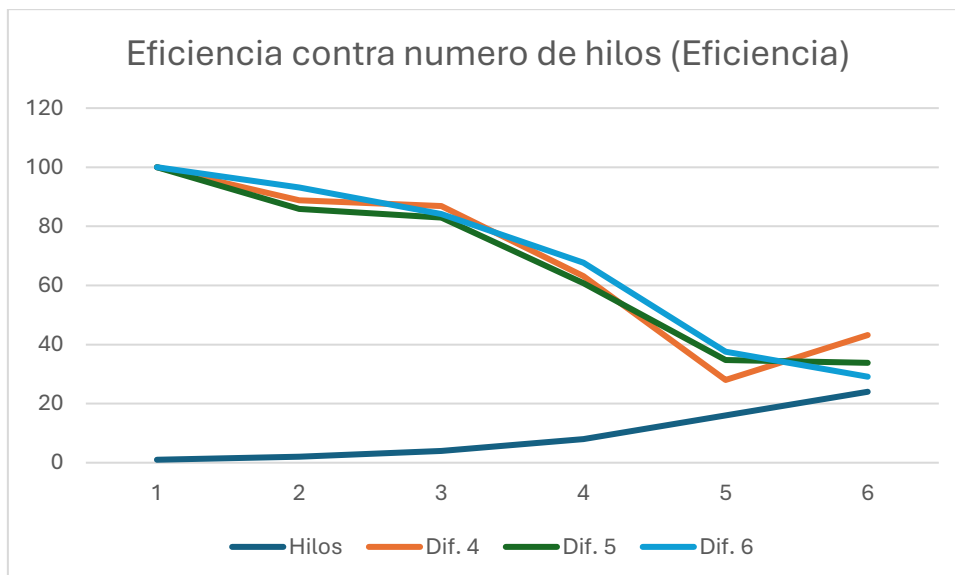
Difficulty	Threads	AvgTimeSeconds	AvgHashRate	Speedup	Efficiency
4	1	0.03	1801636	1	100
4	2	0.0169	3209081	1.776	88.8
4	4	0.0086	6145029	3.474	86.9
4	8	0.0059	8706936	5.05	63.1
4	16	0.0067	9381315	4.482	28
4	24	0.0029	9043484	10.372	43.2
5	1	0.2876	1600217	1	100
5	2	0.1675	2735930	1.717	85.9
5	4	0.0866	5012425	3.321	83
5	8	0.0592	8063582	4.858	60.7
5	16	0.0518	9358135	5.55	34.7
5	24	0.0355	11241119	8.102	33.8
6	1	7.0221	1511493	1	100
6	2	3.7706	2812639	1.862	93.1
6	4	2.0839	5140302	3.37	84.2
6	8	1.297	8093228	5.414	67.7
6	16	1.1679	9275479	6.013	37.6
6	24	1.0048	10618532	6.988	29.1

7.2. Métricas: tiempo, eficiencia y escalabilidad

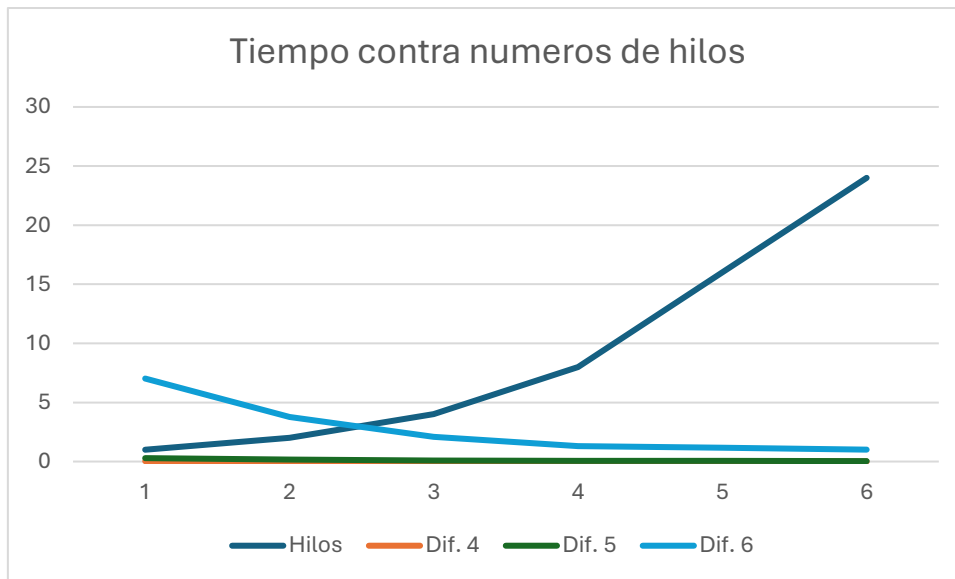
Las métricas registradas son el tiempo de sellado, la velocidad en MHash/s, la aceleración y la eficiencia (aceleración dividida entre el número de hilos). En conjunto muestran cómo evoluciona el rendimiento a medida que se añaden recursos.



La aceleración aumenta con el número de hilos, confirmando la ventaja del paralelismo. El crecimiento es pronunciado hasta los 8 hilos y luego se atenúa, lo que evidencia los rendimientos decrecientes al superar la capacidad real del procesador



La eficiencia parte del 100% con un hilo y descende progresivamente al añadir más, porque los hilos compiten por un número limitado de núcleos físicos. Esto ilustra el concepto de rendimientos decrecientes: cada hilo adicional aporta menos que el anterior.



El tiempo de sellado se reduce drásticamente al añadir hilos: en la dificultad 6 pasa de más de 7 segundos con un solo hilo a aproximadamente 1 segundo con 24 hilos. Esta gráfica traduce la ventaja del paralelismo a términos concretos de tiempo real.

7.3. Análisis de cuellos de botella y limitaciones

Los resultados muestran un patrón característico del paralelismo: la aceleración aumenta al añadir hilos, pero la eficiencia disminuye. Al principio cada hilo aporta casi al máximo (eficiencia cercana al 90–100 %), pero al superar el número de núcleos físicos del procesador los hilos compiten por los mismos recursos y el rendimiento por hilo cae. Este es el principal cuello de botella y es un comportamiento esperado. Otra limitación es que, con dificultades bajas, el trabajo es tan pequeño que el coste de arrancar y coordinar los hilos puede opacar la ventaja del paralelismo; por eso la ventaja se aprecia mejor al aumentar la dificultad.

Análisis por dificultad

Dificultad 4

Con dificultad 4 el trabajo es tan pequeño (el sellado tarda milésimas de segundo) que los resultados son algo irregulares. La aceleración crece bien hasta 8 hilos (5.05x), pero con 16 hilos baja a 4.48x, para luego dispararse a 10.37x con 24 hilos. Esa irregularidad no es un error: cuando el trabajo dura tan poco (0.03 s en secuencial), el costo de arrancar y coordinar los hilos pesa tanto como el trabajo mismo, y pequeñas variaciones del sistema operativo hacen que los números salten. La eficiencia cae rápido (de 100% a 28-43%), lo que confirma que a esta escala de trabajo tan pequeña, tener muchos hilos no compensa.

Dificultad 5

Aquí ya se ve un comportamiento mucho más limpio y coherente, porque el trabajo (0.29 s en secuencial) es lo bastante grande como para que el paralelismo luzca sin tanto ruido. La aceleración sube de forma sostenida: 1.72x con 2 hilos, 3.32x con 4, 4.86x con 8, 5.55x con 16 y 8.10x con 24. La eficiencia va bajando de forma ordenada (100% → 86% → 83% → 61% → 35% → 34%), que es exactamente el patrón que se espera: cada hilo que se añade aporta un poco menos que el anterior.

Dificultad 6

Esta es la dificultad que mejor demuestra el valor del paralelismo, porque el trabajo es grande (7 segundos en secuencial). El sellado pasa de 7.02 segundos con 1 hilo a 1.00 segundo con 24 hilos. La aceleración crece de forma muy consistente (1.86x, 3.37x, 5.41x, 6.01x, 6.99x), y la escalabilidad es la más pareja de las tres dificultades en los primeros tramos: con 8 hilos ya logra 5.41x con una eficiencia todavía buena del 67.7%.

8. Trabajo en Equipo

8.1. Reparto de tareas

Jhonatan, Se encargó del modelo de datos sobre el que se apoya todo el sistema (la representación de las transacciones de energía y de los bloques) así como de la construcción de la cadena de bloques y su mecanismo de verificación de integridad, que es lo que permite detectar cualquier alteración de los datos sellados. Hizo el visor HTML interactivo que muestra la cadena y evidencia de forma visual de cómo una manipulación rompe el sellado.

Daniel se responsabilizó del componente técnico más exigente del proyecto: el motor de paralelismo. Implementó el sellado de bloques en sus distintas versiones (secuencial y paralela) incluyendo las dos estrategias de paralelización que se comparan en el trabajo, junto con los mecanismos de sincronización que coordinan los hilos y evitan condiciones de carrera. También desarrolló el programa principal que integra todos los módulos y expone los distintos modos de ejecución del sistema, además del componente encargado de generar y leer los reportes de energía que alimentan la cadena.

Jonathan, en su rol de líder, asumió la coordinación general del equipo. Tuvo a su cargo la parte de datos y medición del rendimiento. Desarrolló la generación de los datos de energía de la cooperativa y las utilidades de organización de resultados, así como el módulo de evaluación de desempeño (benchmark), que mide y compara el rendimiento de las distintas estrategias con diferente número de hilos y exporta las métricas para su posterior análisis. Finalmente, elaboró las pruebas

del sistema, que verifican el correcto funcionamiento del sellado y de la verificación de la cadena.

8.2. Herramientas de coordinación

Git & GitHub: Control de versiones distribuido con ramas de integración (devep) y trazabilidad de autoría individual obligatoria mediante commits con formato Co-authored-by.

Visual Studio / VS Code: Entorno de desarrollo, perfilado y compilación bajo .NET 8 SDK.

9. Conclusiones

9.1. Principales aprendizajes técnicos

Implementación exitosa del patrón de descomposición exploratoria sobre algoritmos Proof-of-Work.

Dominio de primitivas atómicas no bloqueantes (Interlocked) para evitar los bloqueos pesados (lock), garantizando consistencia sin degradación de rendimiento.

Reducción de la presión sobre el Garbage Collector en entornos multihilo mediante el uso de tipos de valor por referencia (Span<byte>, stackalloc) y formateo UTF-8 directo.

9.2. Retos enfrentados y superados

Condición de carrera al hallar la solución: Resuelto asegurando que solo el primer hilo en encontrar el hash pueda escribir el nonce ganador mediante Interlocked.CompareExchange antes de disparar la cancelación colectiva.

Contención de caché inter-núcleos: Mitigado agrupando las actualizaciones métricas en contadores locales por hilo antes de volcarlas al total compartido.

9.3. Posibles mejoras o líneas futuras

Delegación del cómputo intensivo de hashes a GPU mediante sombreadores de cómputo (DirectCompute/OpenCL) para alcanzar tasas de varios GHash/s.

Incorporación de ajuste dinámico de dificultad algorítmica para estabilizar los tiempos de bloque ante fluctuaciones en la potencia de cómputo de la comunidad.

10. Referencias

Chinta, M. (2024, Mayo 20). *Building a Simple Blockchain in C# with .NET*. Retrieved from Medium: <https://medium.com/codenx/building-a-blockchain-in-net-06f5e3481aab>

He, H. (2024). *Building a Blockchain in .NET Core*. Retrieved from C# Corner: <https://www.c-sharpcorner.com/article/blockchain-basics-building-a-blockchain-in-net-core/>

Microsoft. (2025, Octubre 22). *Task Parallel Library (TPL)*. Retrieved from Microsoft Learn: <https://learn.microsoft.com/en-us/dotnet/standard/parallel-programming/task-parallel-library-tpl>

Microsoft. (n.d.). *SHA256 Class (System.Security.Cryptography)*. Retrieved from Microsoft Learn: <https://learn.microsoft.com/en-us/dotnet/api/system.security.cryptography.sha256?view=net-10.0>

Sarcar, V. (2024). *Parallel Programming with C# and .NET: Fundamentals of Concurrency and Asynchrony Behind Fast-Paced Applications*. APress.

11. Anexos

11.1. Manual de ejecución del Sistema

Comando	Qué hace
dotnet run --project src/SolarMiner	Demo: sella un bloque en secuencial y en paralelo y compara los tiempos.
dotnet run --project src/SolarMiner -- benchmark	Mide el rendimiento con 1, 2, 4 y 8 hilos y genera el CSV de métricas.
dotnet run --project src/SolarMiner -- chain	Construye la cadena, la verifica y genera el visor HTML.
dotnet run --project src/SolarMiner -- reporte	Genera la página de reportes de energía por período y la abre.
dotnet run --project src/SolarMiner -- sellar quincenal	Lee los CSV de energía y sella un bloque por período (flujo completo).
dotnet run --project tests/SolarMiner.Tests	Ejecuta las pruebas del sistema.

11.2. Capturas y pruebas complementarias

Comparativa secuencial vs paralela

```
NODO MINERO – COOPERATIVA DE ENERGÍA SOLAR (Blockchain)
Demostración de paralelismo por descomposición exploratoria

Generando transacciones de energía de la cooperativa...
20 transacciones guardadas en: C:\Users\jhona\source\repos\Proyectofinalparalela\src\SolarMiner\resultados\energia_cooperativa.csv

Primeras transacciones registradas:
Casa-07|APORTA|1.25kWh|2026-08-03T08:00:00
Casa-06|APORTA|2.08kWh|2026-08-03T08:03:00
Casa-08|APORTA|1.54kWh|2026-08-03T08:06:00
Casa-08|APORTA|2.04kWh|2026-08-03T08:09:00
Casa-06|APORTA|2.79kWh|2026-08-03T08:12:00
...

MINADO SECUENCIAL (1 hilo) – dificultad 5 ceros
Nonce encontrado : 457,517
Hash : 0000015201a1653f32e5d5a25ebfd4699b607ca2e38dbf2025bdf1dbe96c61e2
Hashes probados : 457,518
Tiempo : 0.292 s
Velocidad : 1,566 mil hashes/s

MINADO PARALELO (24 hilos) – dificultad 5 ceros
Nonce encontrado : 457,517
Hash : 0000015201a1653f32e5d5a25ebfd4699b607ca2e38dbf2025bdf1dbe96c61e2
Hashes probados : 658,944
Tiempo : 0.073 s
Velocidad : 9,027 mil hashes/s

COMPARATIVA – la ventaja del paralelismo
Tiempo secuencial : 0.292 s
Tiempo paralelo : 0.073 s (24 hilos)
Aceleración : 4.00x (cuántas veces más rápido)
Eficiencia : 16.7% (aprovechamiento por hilo)
```

El HTML interactivo con los “documentos intactos”

Blockchain de la Cooperativa de Energía Solar

Registro sellado de intercambios de energía · Visor de integridad en vivo

Cómo probar la inmutabilidad: edita el valor de cualquier transacción (por ejemplo, cambia los kWh de un bloque). Al soltar, el navegador recalcula los hashes con la misma fórmula que usó el programa. Verás cómo el bloque alterado y **todos los siguientes** se marcan en rojo: la cadena se rompe. Así se demuestra que nadie puede cambiar un dato sellado sin que se detecte.

Restaurar datos originales

Dificultad: 5 ceros

Cadena íntegra: todos los bloques están correctamente sellados y encadenados.

Bloque Génesis

VÁLIDO

Transacciones

Casa-04|APORTA|7.89kWh|2026-08-11T00:00:00

Casa-04|CONSUME|3.86kWh|2026-08-11T00:00:00

Casa-05|APORTA|7.66kWh|2026-08-11T00:00:00

Casa-05|CONSUME|4.75kWh|2026-08-11T00:00:00

Casa-01|APORTA|6.07kWh|2026-08-12T00:00:00

Casa-01|CONSUME|5.00kWh|2026-08-12T00:00:00

Casa-02|APORTA|2.97kWh|2026-08-12T00:00:00

Hash anterior00

Nonce (sello)5070649

Hash de este bloque0000cda060fbc005ed3f69381f695c47f224da29ff84babbb068bcac070a2f

Bloque #1

VÁLIDO

Transacciones

Casa-01|APORTA|0.15kWh|2026-08-16T00:00:00

Casa-01|CONSUME|3.09kWh|2026-08-16T00:00:00

Casa-02|APORTA|4.80kWh|2026-08-16T00:00:00

Casa-02|CONSUME|2.14kWh|2026-08-16T00:00:00

Casa-03|APORTA|4.26kWh|2026-08-16T00:00:00

Casa-03|CONSUME|2.88kWh|2026-08-16T00:00:00

Casa-04|APORTA|0.41kWh|2026-08-16T00:00:00

Casa-04|CONSUME|3.16kWh|2026-08-16T00:00:00

Hash anterior0000cda060fbc005ed3f69381f695c47f224da29ff84babbb068bcac070a2f

Nonce (sello)8568

Hash de este bloque0000093b47ed6e8d2bd40fd8ec3506b0a2ad7b6d4f70e4bcf5b1b410b6b3772

Medicación de aporte de una de las casas

Blockchain de la Cooperativa de Energía Solar

Registro sellado de intercambios de energía · Visor de integridad en vivo

Cómo probar la inmutabilidad: edita el valor de cualquier transacción (por ejemplo, cambia los kWh de un bloque). Al soltar, el navegador recalcula los hashes con la misma fórmula que usó el programa. Verás cómo el bloque alterado y **todos los siguientes** se marcan en rojo: la cadena se rompe. Así se demuestra que nadie puede cambiar un dato sellado sin que se detecte.

Restaurar datos originales

Dificultad: 5 ceros

!!Cadena rota a partir del bloque #0. Un dato fue alterado y el sello ya no coincide.

Bloque Génesis

ALTERADO

Transacciones

Casa-01|APORTA|8.22kWh|2026-08-01T00:00:00

Casa-01|CONSUME|3.82kWh|2026-08-01T00:00:00

Casa-02|APORTA|4.39kWh|2026-08-01T00:00:00

Casa-02|CONSUME|4.13kWh|2026-08-01T00:00:00

Casa-03|APORTA|6.46kWh|2026-08-01T00:00:00

Casa-03|CONSUME|3.23kWh|2026-08-01T00:00:00

Casa-04|APORTA|4.33kWh|2026-08-01T00:00:00

Casa-04|CONSUME|4.93kWh|2026-08-01T00:00:00

Hash anterior

00

Nonce (sello)

5070649

Hash de este bloque

37389344b9e18a8e19364a09fc56927ce7017bd4b52ad03ebc147b1f785b8ae3

Bloque #1

ALTERADO

Transacciones

Casa-01|APORTA|8.15kWh|2026-08-16T00:00:00

Casa-01|CONSUME|3.09kWh|2026-08-16T00:00:00

Casa-02|APORTA|4.80kWh|2026-08-16T00:00:00

Casa-02|CONSUME|2.14kWh|2026-08-16T00:00:00

Casa-03|APORTA|4.26kWh|2026-08-16T00:00:00

Casa-03|CONSUME|2.88kWh|2026-08-16T00:00:00

Casa-04|APORTA|8.41kWh|2026-08-16T00:00:00

Casa-04|CONSUME|3.34kWh|2026-08-16T00:00:00

Hash anterior

37389344b9e18a8e19364a09fc56927ce7017bd4b52ad03ebc147b1f785b8ae3

Nonce (sello)

8568

Hash de este bloque

3cc91b7b7de7534fcab6b831bdc1f1a252759273f483577f28a11a372bd6ff13

11.3. Enlace al repositorio.

<https://github.com/j0nael/Proyectofinalparalela>